

Why the MCP surface is 45 tools, not 289

The design rationale behind the Model Context Protocol layer, the coverage that decision actually costs, and how the server is built.

Scope src/mcp Tools 45 across 5 domains API 289 v1 operations Date 30 September 2026

45 / 289 Tools exposed against documented v1 operations	~383 tok Measured schema cost per tool, re-sent every turn	140 ops In resource families no tool reaches — the real cost
---	--	--

The decision

Exposing all 289 operations was rejected on context budget, not on effort. A tool's JSON schema is re-sent to the model on **every single turn**. At the measured ~383 tokens per tool, a complete surface costs roughly **110 000 tokens** of permanent overhead before the model reads one word of the user's question. Tool-selection accuracy also degrades sharply once dozens of near-identical options are in play — `UpdateRates` , `UpdateRatePlanRates` and `UpdateRatesAcrossRatePlans` are three different endpoints and one human intention.

The second reason is shape. The API is designed for programmers holding state across calls; a model is not. Booking one reservation over raw endpoints is a three-call sequence with ids threaded between them, and a model that stops halfway leaves a half-built booking behind:

endpoint-shaped		task-shaped
GET /booking/v1/offers		booking_create
POST /booking/v1/bookings	->	one call
PUT /reservation-actions/{id}/...		succeeds, or explains why not
^ ids threaded by the model		

The governing rule was therefore: **one tool per question a hotelier asks**, not one per endpoint. `booking_quote_stay` wraps offers, availability and pricing; `operations_day_board` wraps four list calls into what a duty manager wants at 07:00.

THE TRADE, STATED PLAINLY

This buys a usable assistant and pays for it in reach. Roughly half the API is not addressable through MCP. That is a real limitation, not an implementation gap, and it is listed in full below rather than left to be discovered.

What the 45 tools actually reach

MEASURE	COUNT	NOTE
Documented v1 operations in the API	289	All implemented and served over HTTP
Dispatch call sites across the tools	85	Several tools call more than one endpoint
Operations in families the tools touch	149	Upper bound on reach, not actual use
Operations in families with no tool	140	Unreachable through MCP

Where the gaps are

AREA	OPS	CONSEQUENCE
Blocks, block-actions, groups	21	No group or allotment business at all — the largest hole
Authorizations, payment accounts	17	No card authorisation or capture flow
Charge routing	5	Cannot route charges to a company folio
Policies, companies, market segments, promo and corporate codes	~28	Commercial configuration is read-only at best
Property CRUD, settings, languages, age categories	~30	No property setup
Unit attributes, unit actions	6	Partial inventory administration

The `insight` and `setup` domains lean read-only, so a model can generally see this configuration through `insight_property_overview` — it simply cannot change it.

What the split buys

The division into five servers followed the measurement; it was not an a-priori guess. Schema size is the exact serialised `tools/list` payload, so the kilobytes are measured and the token column is that figure at 3.7 chars/token.

SERVER	TOOLS	SCHEMA	~TOKENS	COVERS
<code>booking</code>	13	21.3 KB	5.8k	Quote, book, amend, assign, check in/out, cancel
<code>finance</code>	12	16.4 KB	4.4k	Folios, charges, payments, invoices, sub-ledger
<code>operations</code>	8	10.3 KB	2.8k	Day board, housekeeping, maintenance, night audit
<code>insight</code>	6	7.4 KB	2.0k	Availability, occupancy, ADR/RevPAR, revenue
<code>setup</code>	6	8.3 KB	2.3k	Rooms, rate plans, prices, services, overbooking
<code>all</code>	45	63.7 KB	17.2k	Every domain on one connection

A client mounting a single domain carries about **3.4k tokens instead of 17.2k** — a fifth of the budget. Names are namespaced, so mounting several together never collides.

How it is built

- 1 **`core/dispatch.ts`** — tools do not import the domain layer. They call the real Express application in-process through a synthetic `IncomingMessage` / `ServerResponse` pair, so every route, validator and business rule runs with no socket. Measured at 0.20–0.26 ms. This is what stops an MCP tool and an HTTP client drifting apart in what they validate or permit.
- 2 **`core/errors.ts`** — maps status codes to text that names the recovery tool and is returned as a tool-execution error, never a protocol error, so the model can self-correct: `Reservation 'NOPE-1': not found`. Use `booking_find_reservations` to find valid ids.
- 3 **`core/render.ts`** — aligned tables rather than raw JSON, with empty columns dropped and long lists capped explicitly, so nothing truncated ever looks complete.
- 4 **`core/define.ts`** — the quality gate. `USE WHEN`, `DO NOT USE FOR` and `RETURNS` are required fields on the tool type, so a vague description cannot compile. It also enforces the confirmation gate: declaring `requiresConfirmation` without a `confirm` argument throws at registration and the server refuses to start.
- 5 **`domains/`** — the 45 tools, grouped by job rather than by API, each domain independently mountable.

Safety

Seven operations cannot be undone by calling another tool and are gated behind `confirm: true` — `booking_cancel`, `booking_mark_no_show`, `finance_post_refund`, `finance_cancel_invoice`, `operations_run_night_audit`, `rates_set_prices` and `rates_set_overbooking`. The refusal quotes the real consequence, which is why the check runs after the entity is read:

Not performed. Cancelling SMGHVGFP-1 (Marta Kowalska, 2026-10-01 to 2026-10-03) would charge a cancellation fee of 184.00 EUR and cannot be undone. Call again with `confirm=true` once the user has agreed.

A test asserts the gated set matches the irreversible set exactly, so adding an irreversible tool without a gate fails the build. `destructiveHint` and confirmation are kept distinct: reverting a check-in is destructive but recoverable, so it carries the hint without the gate.

Recommendation

Generating 289 tools would trade a working assistant for nominal completeness and is not advised. Two narrower options close the gaps that matter:

OPTION	COST	EFFECT
Add a <code>groups</code> <code>domain</code> plus routing in <code>finance</code> — roughly 8–10 tools	~3k tok	Closes blocks, allotments, wash and charge routing: the functionality a hotelier would actually miss. Only clients that need it pay. Preferred.
Add one escape hatch — <code>apaleo_request</code> , raw method and path	~400 tok	Reaches the entire long tail of configuration endpoints, but requires the model to know the API shape and bypasses confirmation gating. Should be kept out of the default mount.